

Fondamenti di Informatica T2

Lab06 – MasterMind

Corso di Laurea in Ingegneria Informatica

Anno accademico 2023/2024

Prof. ROBERTA CALEGARI

Prof. AMBRA MOLESINI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)

Obiettivo dell'esercitazione

- Realizzare un'applicazione che simuli il codificatore del noto gioco MasterMind

MasterMind??
Mai sentito nominare..

Nessun problema!
Come al solito, la cosa
migliore è partire dal
DOMINIO



MisterMind: il gioco (1/3)

- Master Mind è un gioco di abilità fra due giocatori, il *codificatore* e il *risolutore*
 - il codificatore inventa una *combinazione di colori* (il codice segreto) che può contenere anche colori ripetuti
 - il risolutore deve indovinarla entro un prefissato *numero massimo di tentativi*.
- A tal fine sono disponibili *piolini colorati* di diversi colori (di norma 6, ma possono essere 8 o più nelle versioni per giocatori esperti)
- La *combinazione* è di norma costituita da 4 colori (che possono essere 5 nelle versioni per esperti)

Colori:        		
	Proposte	Risposte
1	   	☐ ☐
2	   	☐ ☐
3	   	☐ ☐
4	   	☒
5	   	☐
	— — — —	☒ ☒ ☒ ☒

MisterMind: il gioco (2/3)

COME SI GIOCA

- Il risolutore fa un *tentativo (combinazione)*
- Il codificatore **risponde** con una serie di *piolini di risposta* bianchi o neri, disposti a lato:
 - un **piolino nero** per ogni colore indovinato nella giusta posizione
 - un **piolino bianco** per ogni colore presente ma in posizione errata
 - **nessun piolino** per ogni colore non presente nella combinazione segreta
- La *risposta* comprende perciò da 0 a 4 piolini, bianchi e neri
- L'ordine non è significativo: conta unicamente la quantità di piolini neri e bianchi.

Colori:        		
	Proposte	Risposte
1	   	 
2	   	 
3	   	 
4	   	
5	   	
	— — — —	    

MisterMind: il gioco (3/3)

CHI VINCE?

- Vince il risolutore se riesce a dedurre il **codice segreto** entro il **numero massimo di tentativi** previsti; altrimenti, vince il codificatore
- In alcuni tornei si attribuisce un punteggio in base al numero di tentativi effettuati

CURIOSITÀ

- Al posto dei colori si possono usare numeri, e/o al posto dei polini bianchi e neri di risposta si possono usare due simboli diversi (es. "O"/"X") o polini bianchi e rossi
- Sono disponibili vari simulatori online, ad esempio <http://mastermind.it>

Colori:        		
	Proposte	Risposte
1	   	 
2	   	 
3	   	 
4	   	
5	   	
	— — — —	    

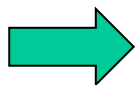
La logica del codificatore

- **Come si sintetizza la risposta?**

- Calcolare il numero di colori nella giusta posizione (i piolini neri) è banale
- Calcolare il numero di colori in posizione errata (i piolini bianchi), meno
→ Attenzione alla possibile presenza di colori ripetuti!

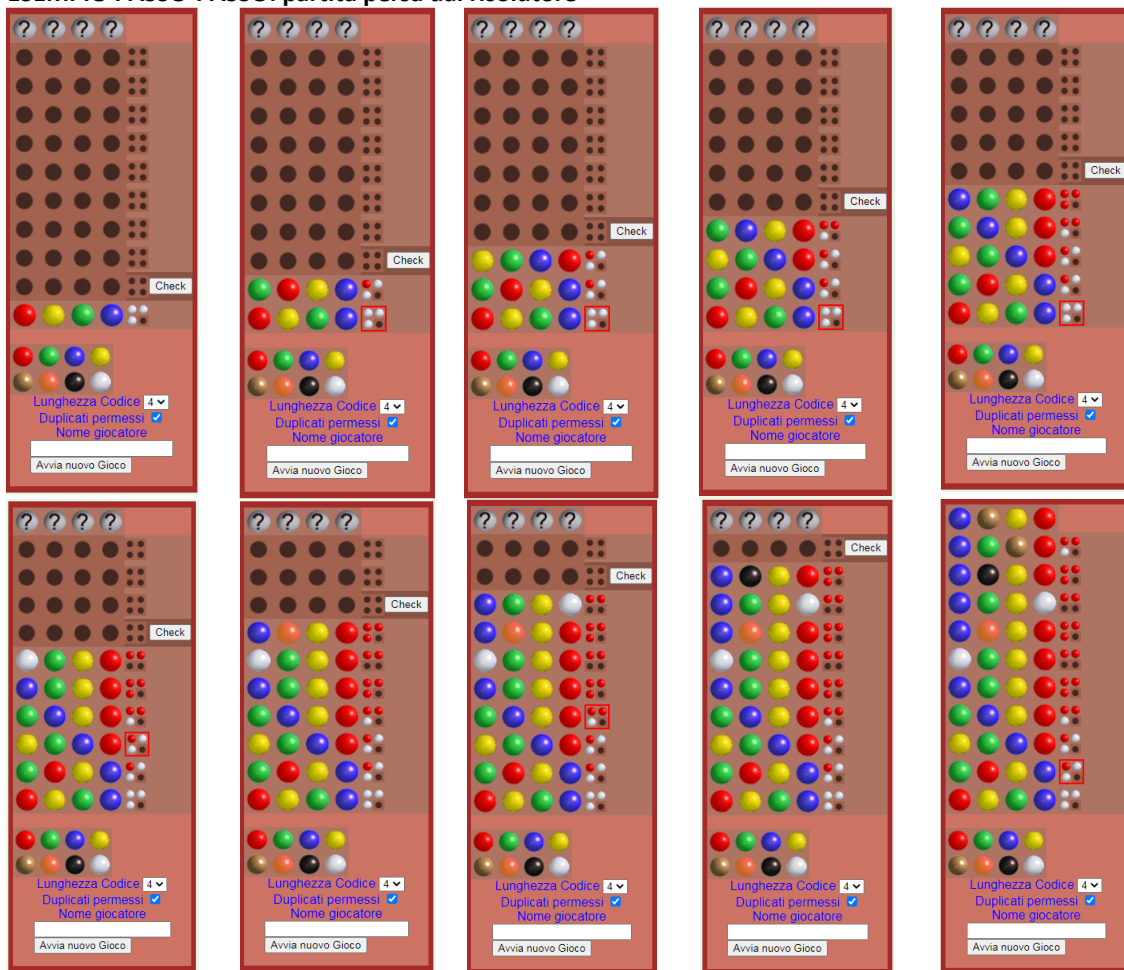
- Possibile approccio

- calcolare preliminarmente il **totale** dei piolini (bianchi + neri) della risposta, **sottraendo** poi quelli neri (facili da calcolare)
- A tal fine sono possibili diverse strategie, quali ad esempio:
 - a) operando su una *copia* della combinazione segreta, considerare un piolo per volta, svuotando via via quelli già considerati, in modo da non contarli più volte
 - b) calcolare preventivamente il *numero di occorrenze di ogni colore* rispettivamente nella combinazione segreta e nel tentativo corrente, agendo poi per differenza.



Esempi di partite (1/2)

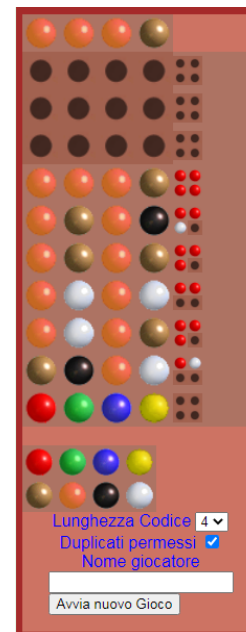
ESEMPIO PASSO-PASSO: partita persa dal risolutore



NB: in questo simulatore i piolini risposta sono bianchi e *rossi* (anziché neri)

Esempi di partite (2/2)

ALTRI ESEMPI: partite vinte dal risolutore



Organizzazione dell'applicazione

- L'applicazione è chiaramente *complicata*: non può essere strutturata come singolo componente
 - sarebbero *troppe funzionalità* in un singolo componente “monolitico”
 - non sarebbe gestibile, né riutilizzabile
- Inoltre, l'applicazione presenta *aspetti chiaramente disgiunti*
 - user interface (UI)
 - organizzazione interna (modello dei dati)
 - coordinamento fra le due
 - ...

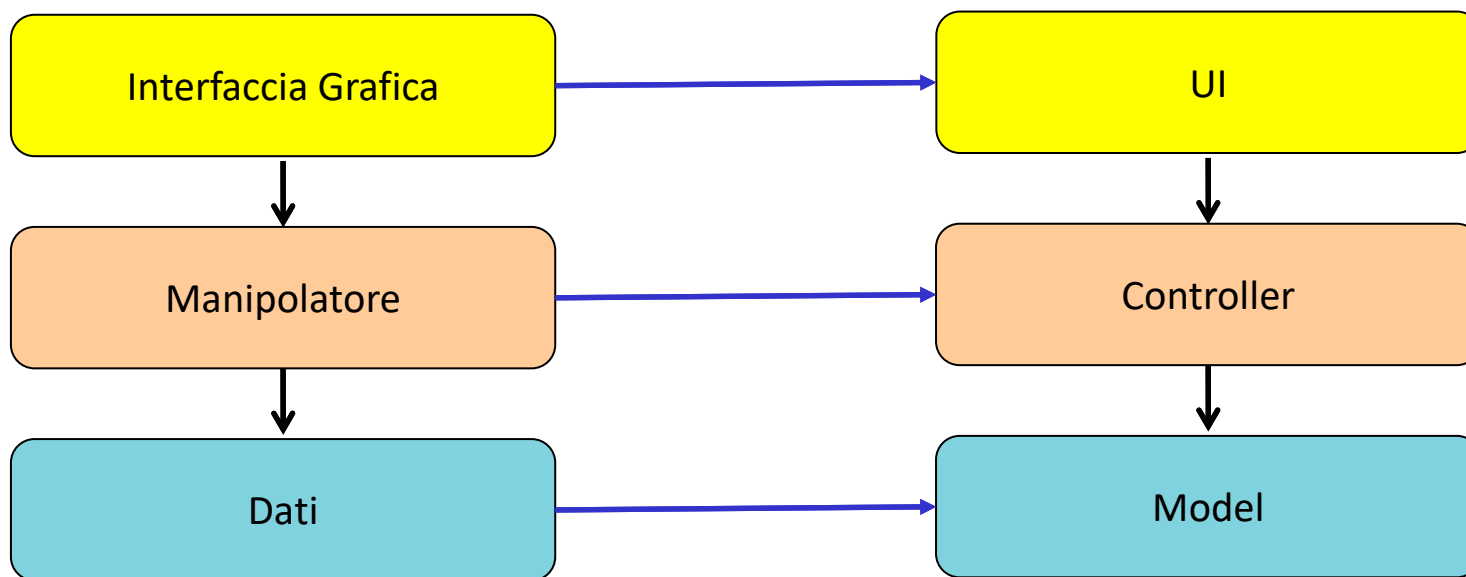
*La struttura è essenziale!
Spesso, più della parte
algoritmica..*

Come organizzare
"bene" un sistema
come questo ?



La struttura dell'applicazione

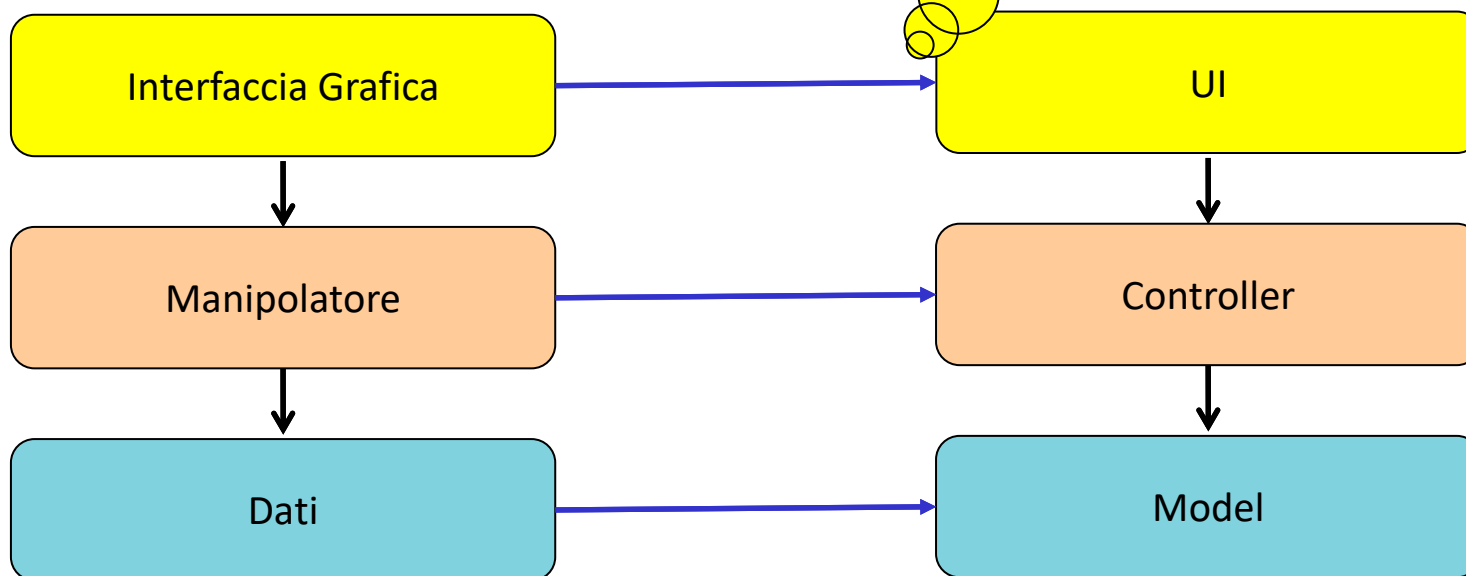
- L'architettura software non è banale, ci sono parecchie entità da modellare e responsabilità diverse da attribuire
- Organizziamo quindi il nostro progetto in diversi package in base alle «diverse funzionalità» di cui si occupano



La struttura dell'applicazione

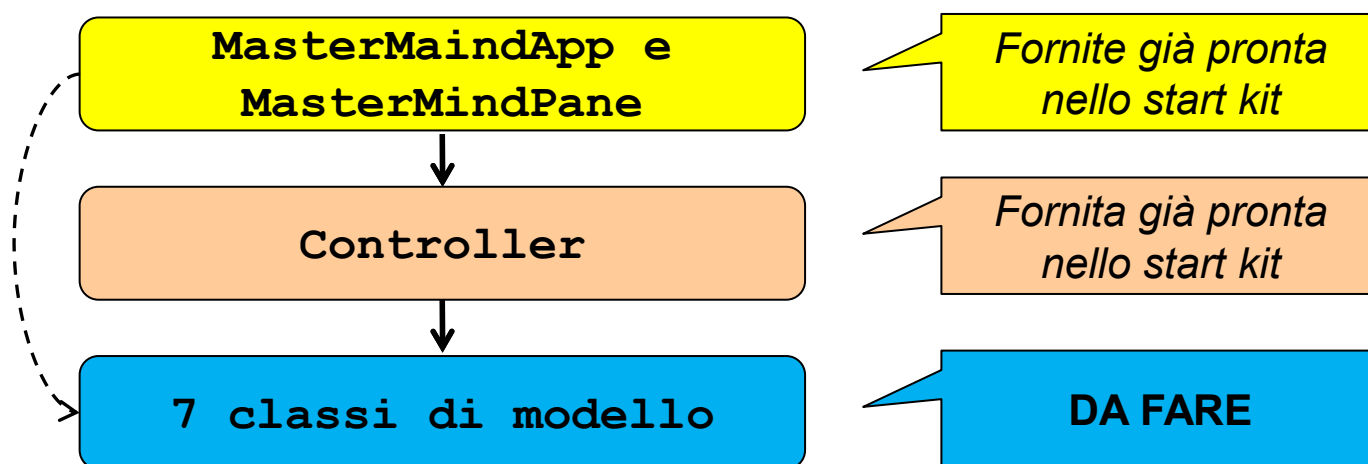
- L'architettura software non è modellare e responsabilità
- Organizziamo quindi il nostro sistema base alle «diverse funzionalità» di cui

Esiste un «design pattern» che risolve esattamente questo problema.. Lo vedremo nella prossima esercitazione



La struttura dell'applicazione

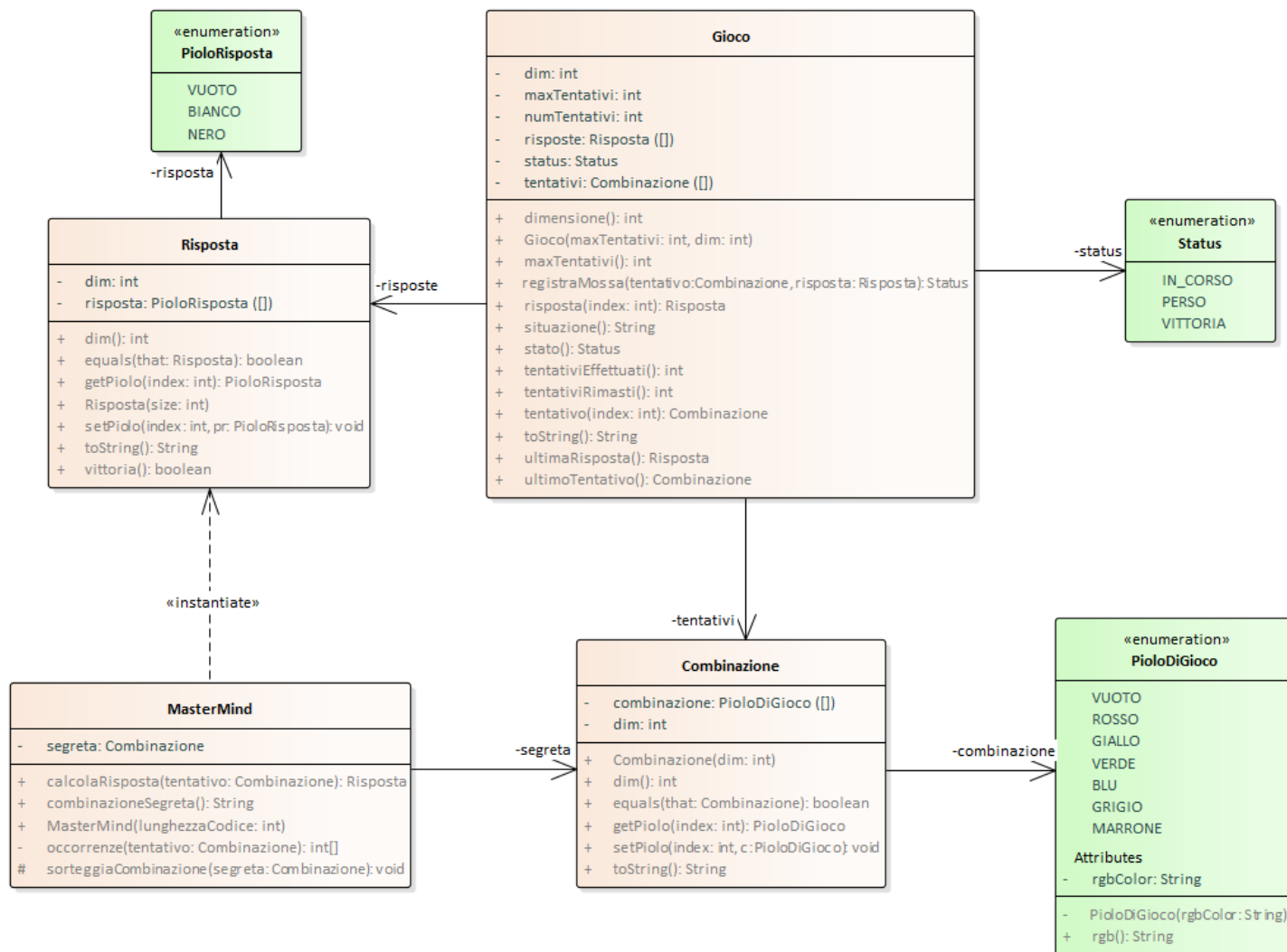
- Nel caso della nostra applicazione MisterMind:
 - **Model:** *svariate classi che memorizzano i dati e lo stato del gioco*
 - **UI:** package **ui** realizzato con JavaFX (*fornito nello start kit*)
 - **Controller:** classe **Controller** (*fornita nello start kit*)



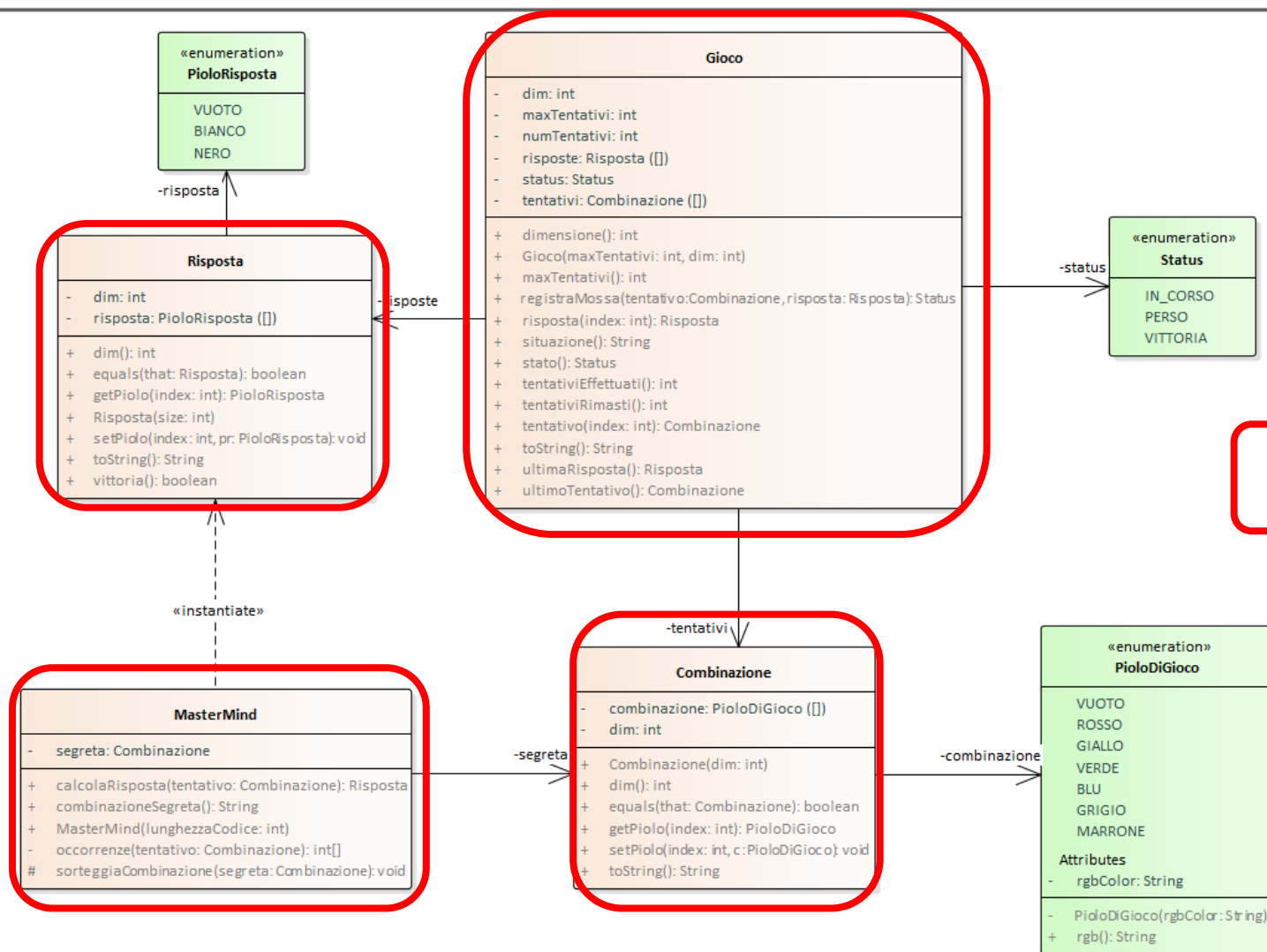
Il Model

- Il model in questa applicazione è più complicato del solito, ci sono parecchie entità da modellare e occorre fare attenzione e chiarire i termini usati nel dominio perché potrebbero essere fuorvianti
 - **Piolo di gioco** (5-6 colori possibili)
 - **Piolo risposta** (due colori, bianco/nero)
 - **Stato** della partita (in corso, vinta, persa)
 - **Combinazione** (il tentativo del risolutore)
 - **Risposta** (del codificatore)
 - Il **Gioco** inteso come struttura (dimensioni, tentativi, risposte, etc.)
 - **MasterMind vero e proprio**, inteso come «cervello» con la logica di gioco

Il modello



Il modello



Da fare

I tre tipi enumerativi

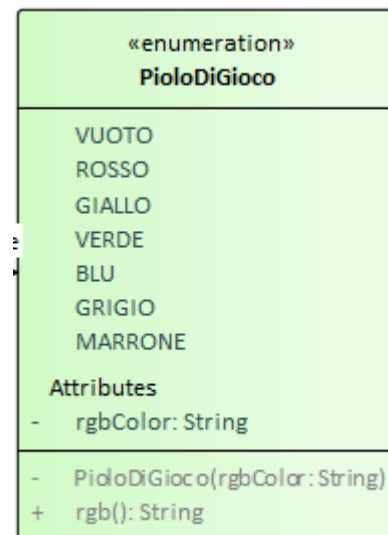
```
package mastermind.model;

public enum PioloDiGioco {
    VUOTO("#808080"), ROSSO("#FF0000"), GIALLO("#FFFF00"), VERDE("#00FF00"),
    BLU("#00FFFF"), GRIGIO("#C0C0C0"), MARRONE("#800000");

    private String rgbColor;

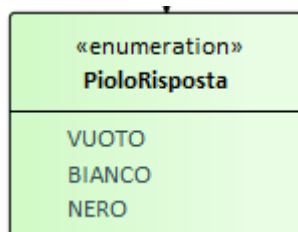
    private PioloDiGioco(String rgbColor) {
        this.rgbColor=rgbColor;
    }

    public String rgb() {
        return rgbColor;
    }
}
```

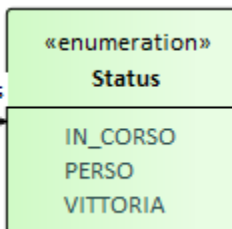


```
package mastermind.model;

public enum PioloRisposta {
    VUOTO, BIANCO, NERO;
}
```



Già pronti nello startkit



```
package mastermind.model;

public enum Status {
    IN_CORSO, PERSO, VITTORIA;
}
```

Combinazione

- Rappresentazione interna: un array di «opportuni» pioli (i PioliDiGioco)
 - il costruttore riceve la dimensione (numero di pioli), crea l'array interno e ne riempie le celle alla costante VUOTO
 - appositi accessor impostano/recuperano il singolo piolo e la dimensione fisica
 - un'apposita *toString* sintetizza una stringa stampabile che mostri la combinazione
 - tale *toString* è usata anche dopo, in altre classi
 - concatena, tramite uno `StringJoiner`, i vari colori separandoli con virgola, senza aggiungere alcun prefisso né suffisso (né spazi intermedi)
- ESEMPIO: per la combinazione «rosso, giallo, verde, blu» viene emessa la stringa **"ROSSO,GIALLO,VERDE,BLU"**

Combinazione	
-	<code>combinazione: PioloDiGioco ([])</code>
-	<code>dim: int</code>
+	<code>Combinazione(dim: int)</code>
+	<code>dim(): int</code>
+	<code>equals(that: Combinazione): boolean</code>
+	<code>getPiolo(index: int): PioloDiGioco</code>
+	<code>setPiolo(index: int, c: PioloDiGioco): void</code>
+	<code>toString(): String</code>

Risposta

- Rappresentazione interna: un array di «opportuni» pioli (i PioliRisposta)
 - struttura molto simile a Combinazione
 - il costruttore riceve la dimensione (numero di pioli), crea l'array interno e ne riempie le celle alla costante VUOTO
 - appositi accessor impostano/recuperano il singolo piolo e la dimensione fisica
 - un'apposita *toString* sintetizza una stringa stampabile che mostri la risposta (formato analogo al caso precedente)
 - il metodo boolean *vittoria* restituisce *true* se la risposta è *composta da tutti e soli piolini neri*

Risposta	
-	dim: int
-	risposta: PioloRisposta ([])
+	dim(): int
+	equals(that: Risposta): boolean
+	getPiolo(index: int): PioloRisposta
+	Risposta(size: int)
+	setPiolo(index: int, pr: PioloRisposta): void
+	toString(): String
+	vittoria(): boolean

Gioco

- Rappresenta il piano di gioco

Gioco
- dim: int - maxTentativi: int - numTentativi: int - risposte: Risposta ([]) - status: Status - tentativi: Combinazione ([])
+ dimensione(): int + Gioco(maxTentativi: int, dim: int) + maxTentativi(): int + registraMossa(tentativo: Combinazione, risposta: Risposta): Status + risposta(index: int): Risposta + situazione(): String + stato(): Status + tentativiEffettuati(): int + tentativiRimasti(): int + tentativo(index: int): Combinazione + toString(): String + ultimaRisposta(): Risposta + ultimoTentativo(): Combinazione



Gioco: descrizione generale

- Contiene gli elenchi di tentativi e risposte
 - sa quanti tentativi sono ammessi (al massimo)
 - mantiene l'indice del tentativo corrente
 - offre metodi per recuperare l'i-esimo tentativo / l'i-esima risposta, lo stato, l'ultimo tentativo / l'ultima risposta
 - offre una **toString** che sintetizza la stringa stampabile desiderata
 - offre il metodo **registraRisposta** che aggiunge agli elenchi interni una nuova coppia (tentativo, risposta)

Gioco: semantica metodi

- Il **costruttore** riceve il numero massimo di tentativi e la dimensione, recuperabili tramite gli accessor `maxTentativi` e `dim`
- **tentativiEffettuati** e **tentativiRimasti** restituiscono il numero di tentativi effettuati e rimasti, rispettivamente
- **tentativo(int i)** e **risposta(int i)** restituiscono rispettivamente la combinazione proposta dal concorrente / la risposta data dal codificatore nel tentativo *i*-esimo
- **ultimoTentativo** e **ultimaRisposta** costituiscono due scorciatoie per i metodi precedenti, specializzate per l'ultimo tentativo
- **registraMossa** aggiorna lo stato registrando la coppia (tentativo, risposta) ricevuta come argomento, restituendo lo *Status* aggiornato
- **stato** restituisce lo *Status* corrente
- **situazione** sintetizza e restituisce la stringa (multi-linea) che descrive tutti i tentativi finora effettuati, con le relative risposte (*dettagli nella prossima slide*)
- **toString** combina in una singola stringa omnicomprensiva la situazione e lo stato ottenuti dai metodi precedenti (*dettagli nella prossima slide*)



- ### Esempio:

3) BLU,BLU,BLU,BLU\t\t\tNERO



Gioco: formato stringhe

- **toString** combina in una singola stringa omnicomprensiva la situazione e lo stato ottenuti dai metodi precedenti, come segue:

Situazione:

1) *colore,colore,colore,colore\t\tpiolo,piolo,piolo,piolo*

2) *colore,colore,colore,colore\t\tpiolo,piolo,piolo,piolo*

...

Gioco: *status*

Esempio:

Situazione:

1) ROSSO,GIALLO,VERDE,BLU\t\tBIANCO,BIANCO,NERO,NERO

2) ROSSO,BLU,VERDE,GIALLO\t\tNERO,BIANCO,NERO,NERO

Gioco: IN_CORSO

MasterMind

- Rappresenta il «cervello» del gioco
- Sorteggia e mantiene il codice segreto (da indovinare)

MasterMind	
-	segreta: Combinazione
+	calcolaRisposta(tentativo: Combinazione): Risposta
+	combinazioneSegreta(): String
+	MasterMind(lunghezzaCodice: int)
-	occorrenze(tentativo: Combinazione): int[]
#	sorteggiaCombinazione(segreta: Combinazione): void

MasterMind

- Incapsula la logica per sintetizzare la risposta a un tentativo
 - il **costruttore** riceve la lunghezza del codice da generare, alloca il codice segreto (una *Combinazione*) e ne sorteggia il contenuto (tramite il metodo protetto ausiliario **sorteggiaCombinazione**)
 - il metodo **combinazioneSegreta** restituisce il codice segreto come *stringa*
 - il metodo **calcolaRisposta** sintetizza e restituisce la *Risposta* da dare a un certa *Combinazione*
 - **Calcolo piolini neri** → confronto tentativo/risposta, uno ad uno
 - **Calcolo piolini bianchi** → differenza fra occorrenze e piolini neri
 - include due *metodi privati* ausiliari
 - **sorteggiaCombinazione**, usato solo nel costruttore, che popola di colori la *Combinazione* ricevuta come argomento
 - **occorrenze**, usato solo in **calcolaRisposta**, che restituisce un array di interi che specifica quante occorrenze di ciascun colore ci siano in una data *Combinazione*



MasterMind: semantica metodi

- In particolare:

- **sorteggia***Combinazione* sorteggia una *Combinazione* mediante la generazione casuale di numeri nel range 1..max

ATTENZIONE: l'enumerativo *PioloDiGioco* ha anche il valore VUOTO, che non è un colore usabile nella combinazione del codificatore. Di ciò occorre tenere conto nel calcolo del *max nel* range sopra.

ESEMPIO: con 6 colori + vuoto, i valori dell'enumerativo sono 7 (da 0 a 6), ma i valori sorteggiabili come colori devono essere nel range 1..7

- **occorrenze** restituisce un array di interi che specifica quante occorrenze di ciascun colore ci siano in una data *Combinazione*

ESEMPIO: per la combinazione «rosso, rosso, giallo, verde» e colori enumerati nell'ordine «vuoto, rosso, giallo, verde, blu, grigio, marrone», viene restituito l'array [0,2,1,1,0,0,0]

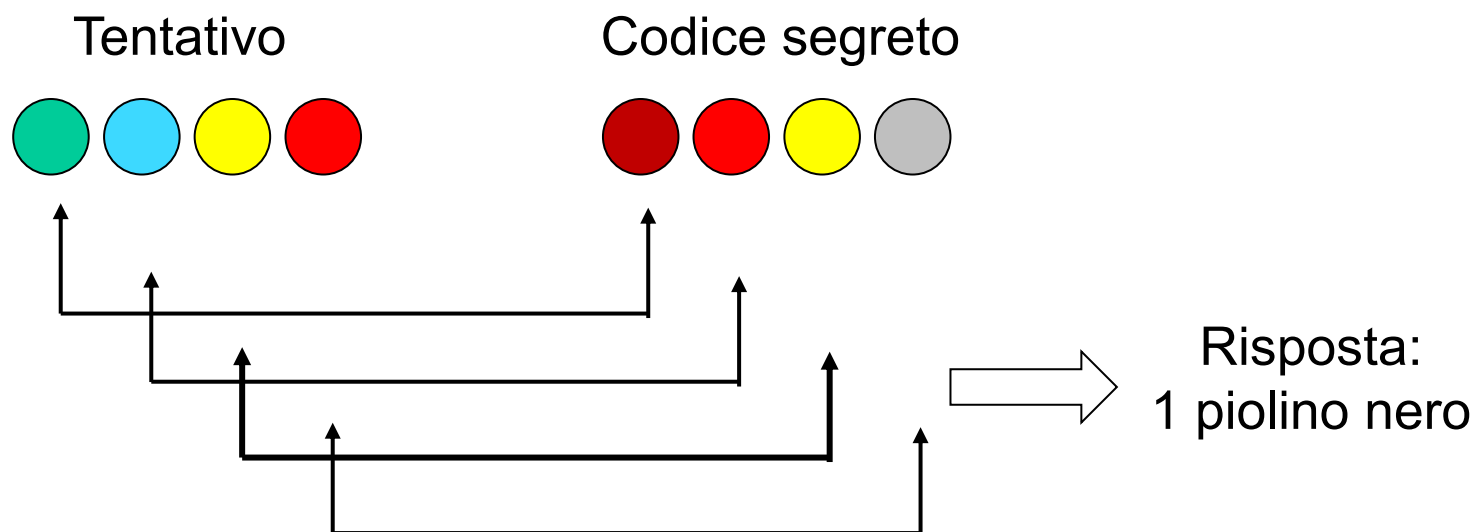


MasterMind: formato stringhe

- L'unico metodo che sintetizza stringhe è `combinazioneSegreta`
 - esso si limita in realtà a richiamare la `toString` di *Combinazione*
 - La `toString` di *Combinazione*, come già detto:
 - concatena, tramite uno `StringJoiner`, i vari colori separandoli con virgola, senza aggiungere alcun prefisso né suffisso (né spazi intermedi)
- ESEMPIO: per la combinazione «rosso, giallo, verde, blu» emette la stringa
`"ROSSO,GIALLO,VERDE,BLU"`

Logica: piolini neri

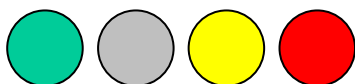
- Calcolo piolini neri → confronto tentativo/risposta, uno ad uno
 - Si procede in ordine confrontando i piolini di egual posizione
 - Se sono uguali, c'è un piolino nero in più nella risposta; altrimenti no



Logica: piolini bianchi (1/5)

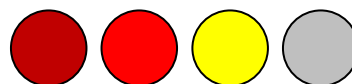
- Calcolo piolini bianchi → **differenza fra occorrenze e piolini neri**
 - Occorre contare le occorrenze di ogni colore nel tentativo e nella risposta
 - Per ogni colore, si prende il MINIMO fra tentativo e risposta: il risultato è il totale di piolini (neri+bianchi) per quel colore nella risposta
 - Per avere i bianchi, occorre quindi sottrarre i piolini neri

Tentativo



Colori	Occorrenze
Rosso	1
Giallo	1
Verde	1
Blu	0
Grigio	1
Marrone	0

Codice segreto

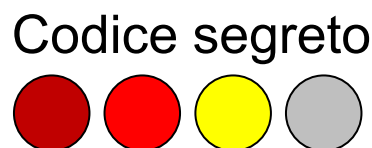
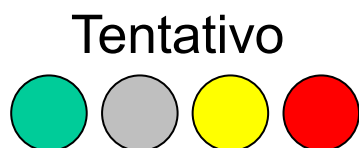


Colori	Occorrenze
Rosso	1
Giallo	1
Verde	0
Blu	0
Grigio	1
Marrone	1

Min
1
1
0
0
1
0

Logica: piolini bianchi (2/5)

- Calcolo piolini bianchi → **differenza fra occorrenze e piolini neri**
 - Occorre contare le occorrenze di ogni colore nel tentativo e nella risposta
 - Per ogni colore, si prende il MINIMO fra tentativo e risposta: il risultato è il totale di piolini (neri+bianchi) per quel colore nella risposta
 - Per avere i bianchi, occorre quindi sottrarre i piolini neri



MA c'era già
1 piolino nero:
quindi, i piolini
bianchi sono 2



Risposta:
3 piolini complessivi
(fra bianchi e neri)

Min
1
1
0
0
1
0

Logica: piolini bianchi (3/5)

- Altro esempio con colori ripetuti *nel tentativo*

Tentativo		Codice segreto		
   		   		
Colori	Occorrenze	Colori	Occorrenze	Min
Rosso	1	Rosso	1	1
Giallo	2	Giallo	1	1
Verde	0	Verde	0	0
Blu	0	Blu	0	0
Grigio	1	Grigio	1	1
Marrone	0	Marrone	1	0

Poiché i minimi non cambiano, la risposta non varia: 

Logica: piolini bianchi (4/5)

- Altro esempio con colori ripetuti *nel codice*

Tentativo		Codice segreto		
   		   		
Colori	Occorrenze	Colori	Occorrenze	Min
Rosso	1	Rosso	1	1
Giallo	1	Giallo	2	1
Verde	1	Verde	0	0
Blu	0	Blu	0	0
Grigio	1	Grigio	1	1
Marrone	0	Marrone	0	0

Anche in questo caso, poiché i minimi non cambiano, la risposta non varia:



Logica: piolini bianchi (5/5)

- Altro esempio con colori ripetuti *nel codice e nel tentativo*

Tentativo		Codice segreto		
   		   		
Colori	Occorrenze	Colori	Occorrenze	Min
Rosso	1	Rosso	1	1
Giallo	2	Giallo	2	2
Verde	0	Verde	0	0
Blu	0	Blu	0	0
Grigio	1	Grigio	1	1
Marrone	0	Marrone	0	0

Stavolta i minimi sono cambiati: la risposta deve contenere 4 piolini. Poiché i neri sono 1, gli altri 3 devono essere bianchi



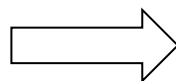
Problema: la tabella delle occorrenze

- Come rappresentare nel codice la tabella delle occorrenze?
 - Servirebbe una struttura dati «tabella» → ce l'avremo, in futuro!
 - Poiché per ora non ce l'abbiamo, o la progettiamo, o.. facciamo senza

Riflettiamo: i colori sono enumerativi, quindi di fatto associati implicitamente a un intero (il loro *ordinal*)

Perciò, la prima colonna della tabella è «quasi» un indice numerico
→ può facilmente diventare l'indice di un array 😊

Colori	Occorrenze
Rosso	1
Giallo	2
Verde	0
Blu	0
Grigio	1
Marrone	0



	1	2	0	0	1	0
--	---	---	---	---	---	---

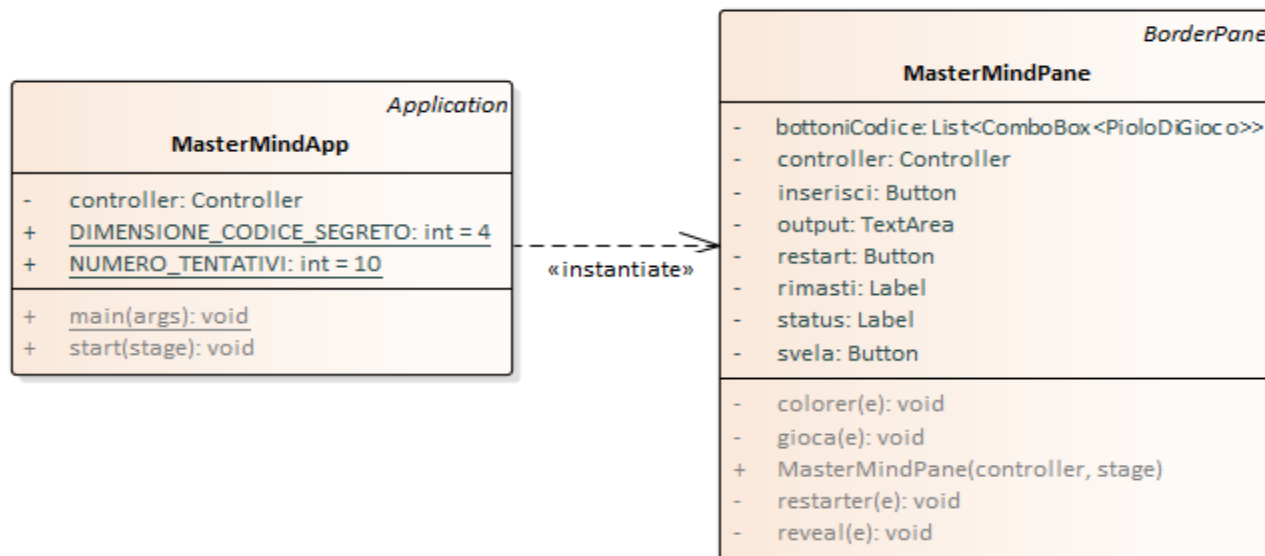
Indice: colore.ordinal()
(0 = vuoto, etc.)

Il Controller

- È il «grande burattinaio» dell'applicazione
- Riceve le richieste dalla View e manipola/interroga le classi del model per ottenere le informazioni da passare alla View
- È già pronto nello Start kit nell'omonimo package

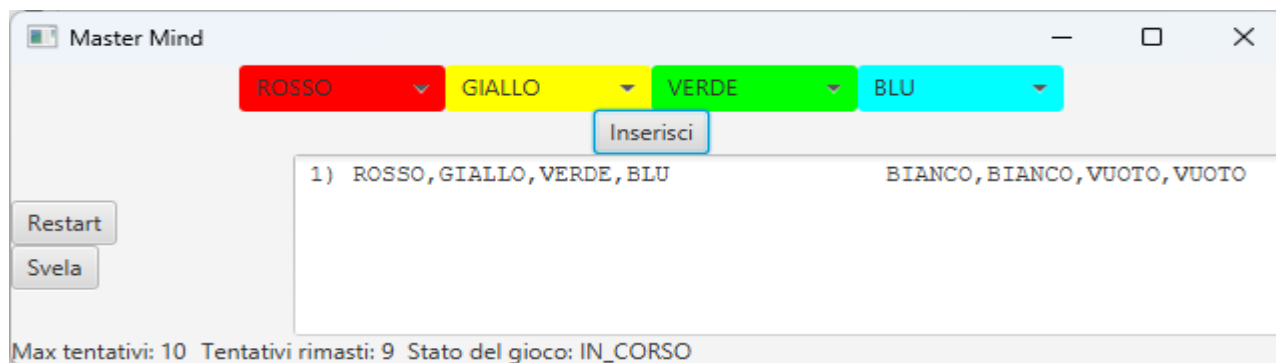
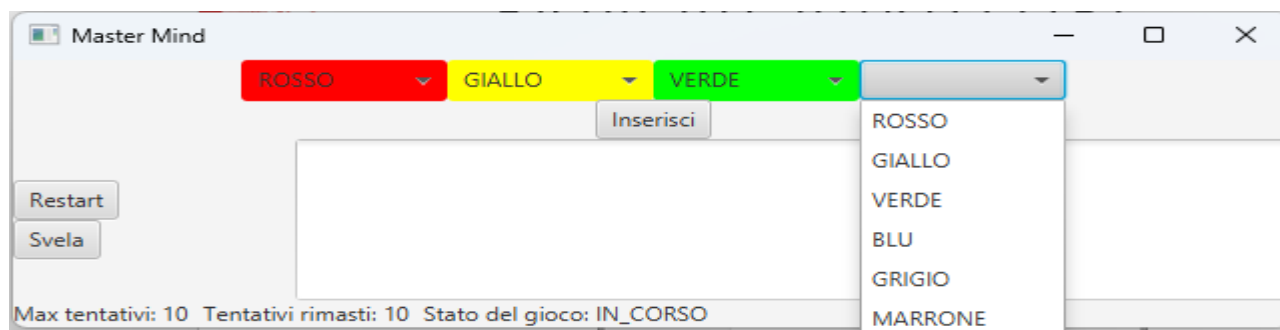
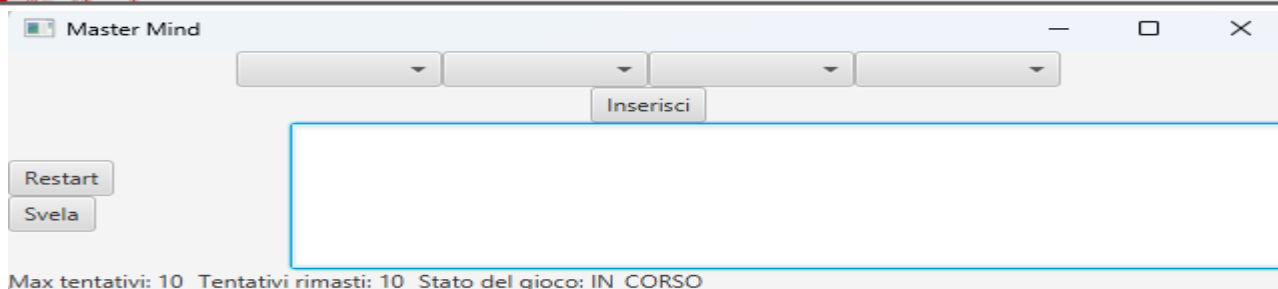
Controller
- brain: MasterMind - dim: int - gioco: Gioco - maxTentativi: int
+ <u>alert(title:String,headerMessage:String,contentMessage:String): void</u> + combinazioneSegreta(): String + Controller(maxTentativi: int, dim: int) + dimensioneCodice(): int - init(maxTentativi: int, dim: int): void + maxTentativi(): int + restart(): void + situazione(): String + status(): Status + tenta(tentativo: Combinazione): Status + tentativiEffettuati(): int + tentativiRimasti(): int

La UI

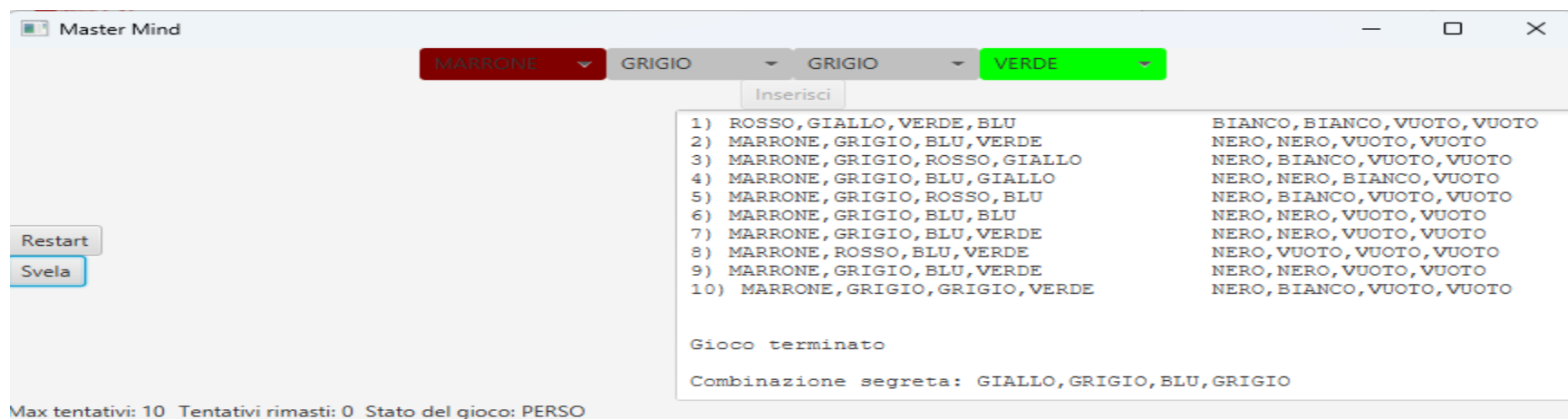
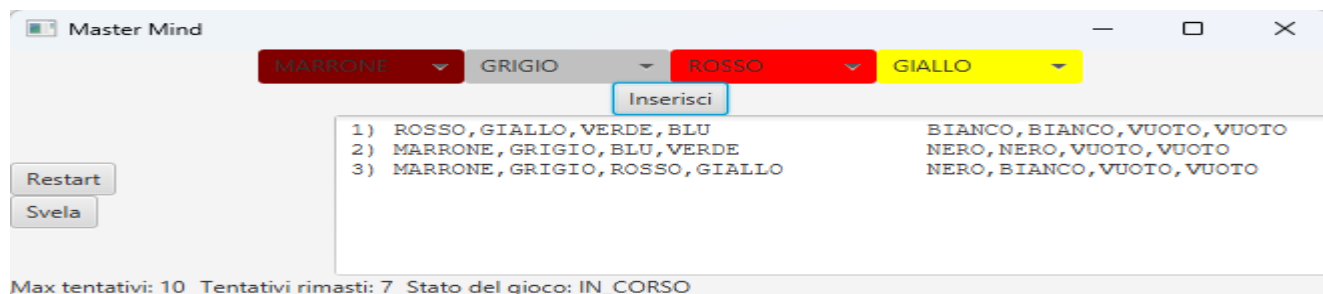
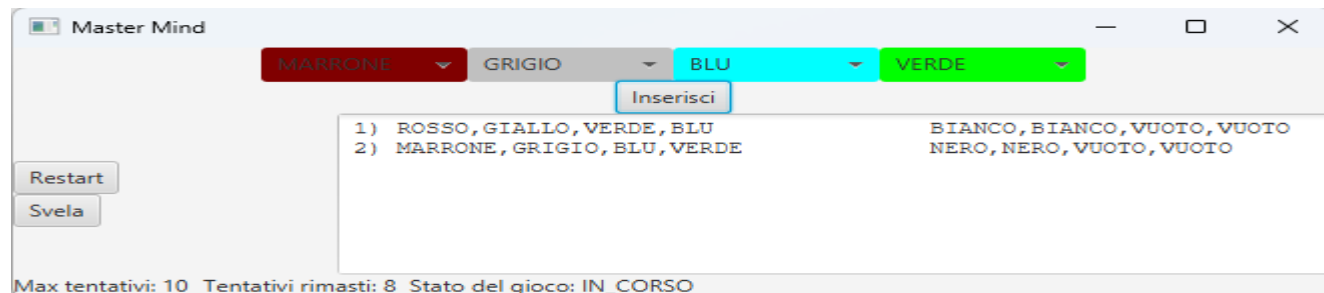


- MasterMainApp e MasterMindPane rappresentano *la ui* nella tecnologia JavaFx
- Classi già pronte nello Start kit (package **ui**)

La UI (1/3)



La UI (2/3)

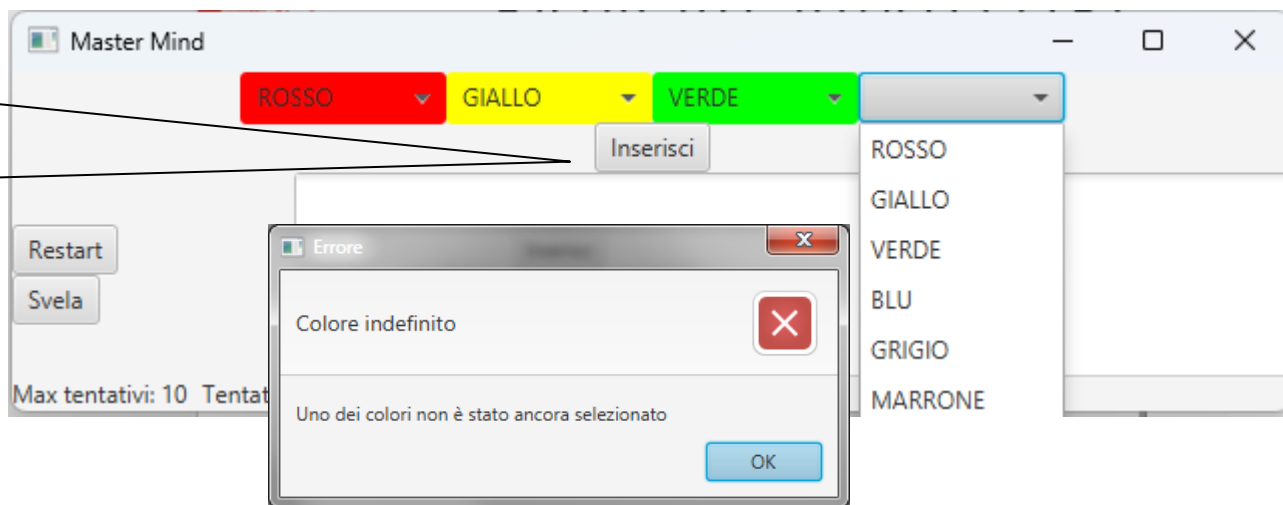


La UI (3/3)

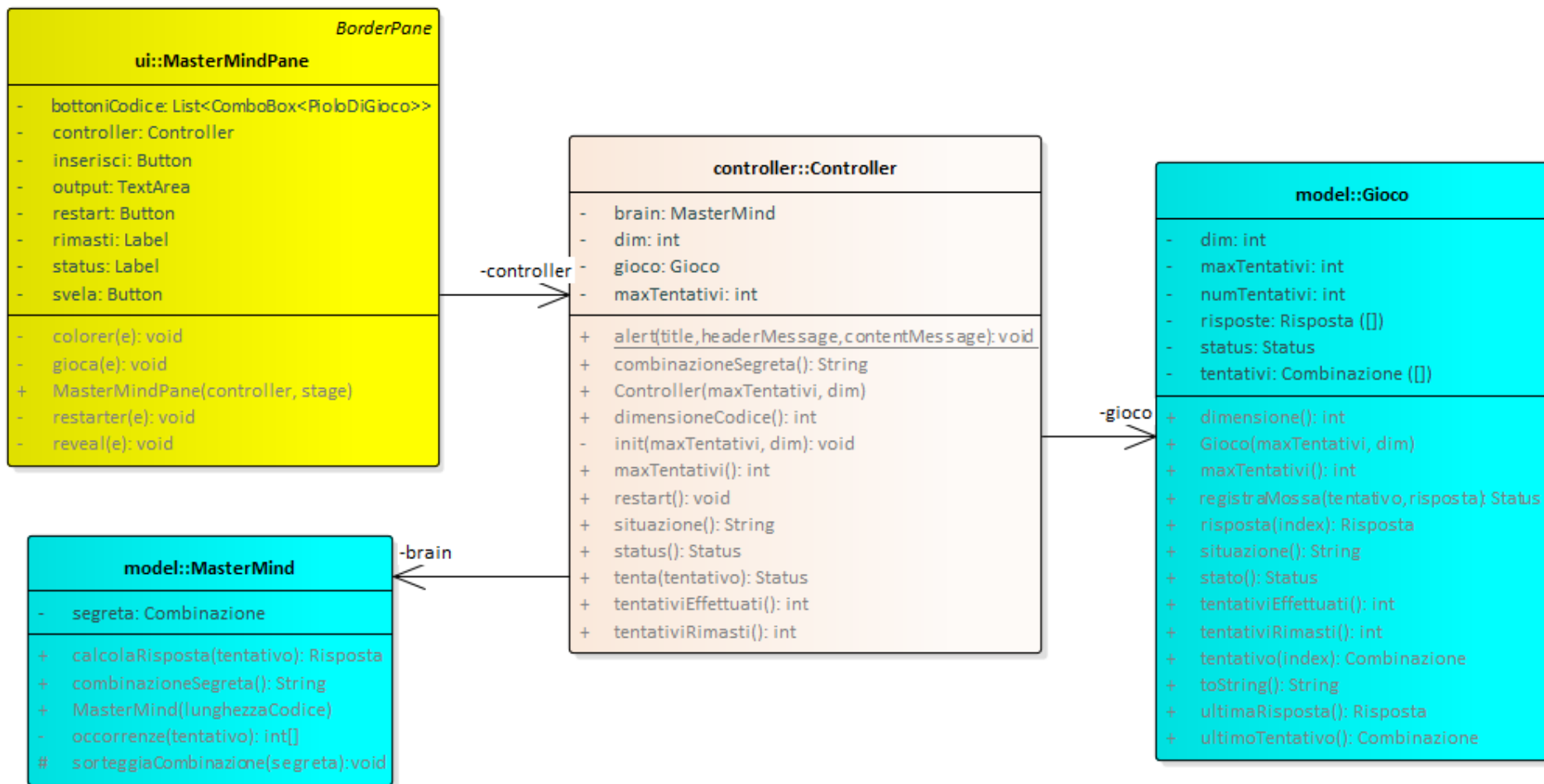
Il pulsante SVELA permette di mostrare la combinazione segreta



Se si preme INSERISCI prima di aver scelto tutti i colori, compare un messaggio d'errore



Architettura (quasi) completa



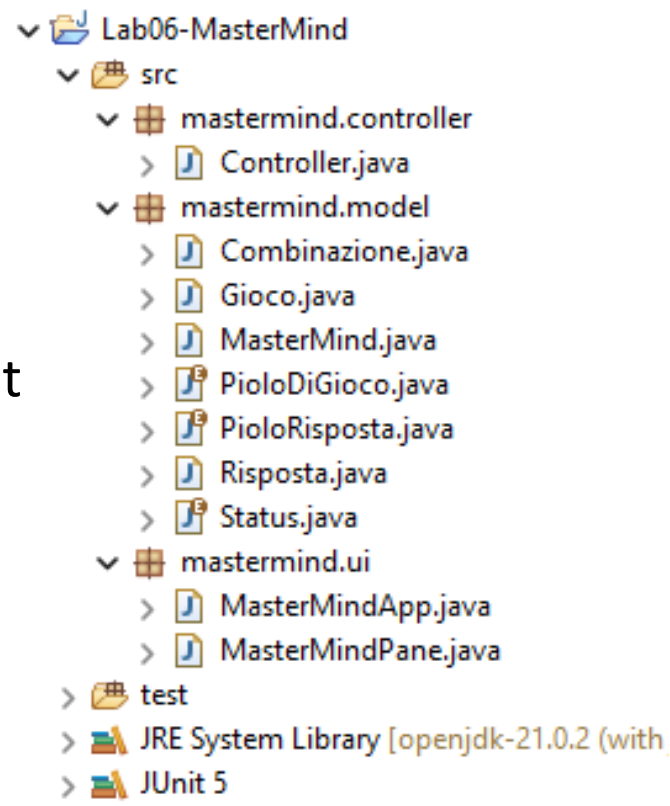
Package & Start Kit

- Classica organizzazione dell'ambiente di lavoro con due source folder per sorgenti e test.

- Organizzazione in package:

- Package model (da fare)
- Package controller già pronto nello startkit
- Package ui (la view) fornito nello start kit

N.B. In laboratorio è già presente una versione di Java con supporto alla libreria JavaFX. Per l'esecuzione basta fare il run sulla classe MasterMindApp



Package model

- **Le classi del model fornite già pronte**

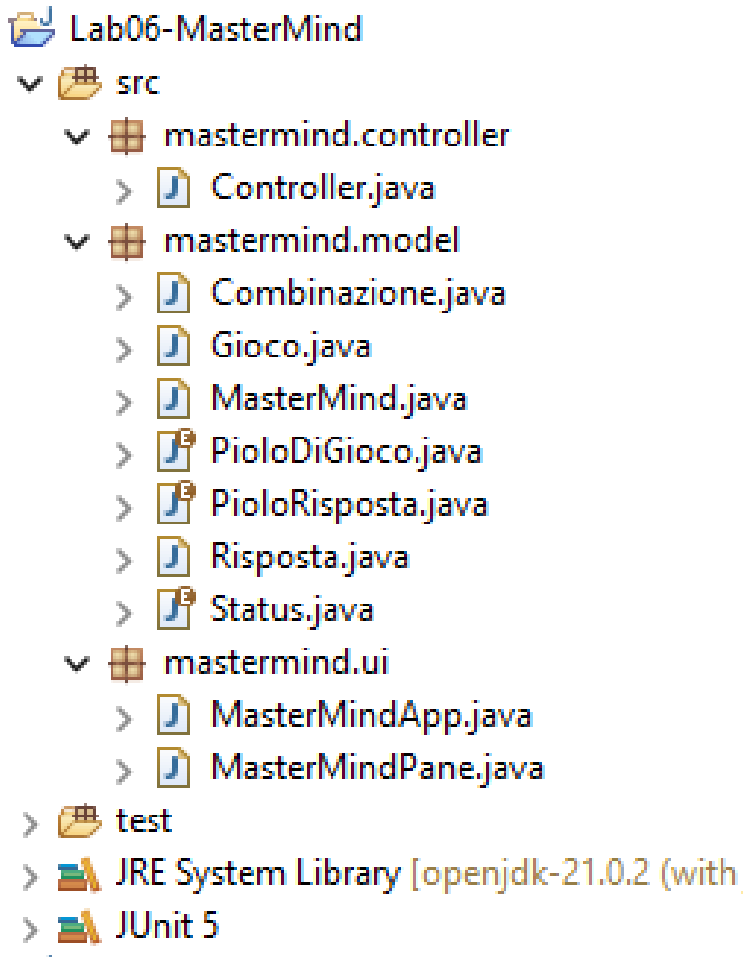
- Piolo di gioco: un enumerativo con 7 colori
- Piolo risposta: un enumerativo con 2 colori
- Status: un enumerativo con 3 stati

- **Le classi da fare**

- **Combinazione** e **Risposta** (quasi identiche)
- **Gioco**
- **MasterMind**

- **Test, test, test!**

- Ci sono un sacco di test già pronti
- Deve funzionare TUTTO!



Hey!



**KEEP
CALM
AND
HAPPY
CODING**